Recursion flow: when recursion happens the function calls are stored in a stack.

main()
↓
factorial(4)
↓
factorial(3)
↓
factorial(2)
↓
factorial(1)

due to recursion
- code become smaller and cleaner
- Good for tree and graph Problem
- Useful in devide and conquer Algo

Cons
- use more memory (stack)
- Can be slower than loop.
- Risk of stack overflow

Problem statement: Reverse a number using recursion.
Sample input: 1234
" output: 4321

Ans
1234 → input
↓
Break numbers (1234) % 10 ⇒ (Remainder = 4)
(1234)/10 ⇒ new num = 123.4 → int
⇒ 123.

$\frac{1}{10}$ (2) ← 12 and (3) ← Same Process

↳ print cout << "number" = new num

4321

Day-11 (~~Sample~~ Method Overloading)

function overloading means multiple function having the same name but different parameters.

Why do we need function overloading?
- write a clean code.
- use same function name for similar task
- improve readability.

Like
- add two int
- add two float
- add three numbers.

Problem statement: Array Analyzer using Pointer Arithmetic.

① take a size of array from user.
② take array elements from user.
③ using Pointer arithmetic (NOT arr[i] directly).
- Print all elements.
- Find sum of elements.
- Find maximum elements.
- Find minimum element.

④ Print memory address of each element.

⚠ use pointer arithmetic like *(Ptr+i) instead of arr[i]

Day-10 Recursive function)

A recursive function is a function that calls itself to solve a problem

Instead of solving the whole problem, recursive function break the ~~function~~ problem into small parts.

Every recursive function have.

⊙ Base case
The condition where the function stops to call itself

⊙ Recursive case
The part where function call itself again.

If there is no base case, then program will run infinite times. and crash.

Syntax

```
sum_type func_name (Parameter) {
    if (base_condition) {
        return value;
    }
    return function_name (smaller problem);
}
```

int sum(int a, int b) —
float sum(int a, int b); // Error.

} Because the compiler do not decide which function to call.

Practice Problems: Smart Number Analyzer.

- If one parameter given find even, odd.
- If two parameter given find max, min, which is greater.
- Prime number in range.

OOPS

Examples: (basic function overloading)

```
#include<iostream>
using namespace std;
int add(int a, int b){
    return a+b;
}
float add (float a, float b){
    return a+b;
}
int main() {
    cout << add(5,2)<<endl ;   // call int version
    cout << add (2.5f, 3.5f);  // call float version
    return 0;
}
```

//

Different number of parameters

```
int sum( int a, int b){
    return a+b;
}
int sum (int a, int b, int c){
    return a+b+c;
}
```

Function overloading Rules:

① Number of parameters

sum(int a, int b)

sum(int a, int b, int c)

② Type of parameters

sum (int a, int b)

sum (float a, float b)

③ Order of parameters

sum(int a, float b)

sum(float a, int b)